SQL DATA ENGINEERING TRAINING MANUAL

PostgreSQL | Banking & Financial Systems

Author: Gospel Orok

Version: 1.0

Date: 09 January 2026

Audience: Beginner → Intermediate learners targeting Data Engineering roles
Database: PostgreSQL
Goal: Build strong, production-ready SQL skills for ETL, analytics, and data warehousing

How to Use This Training Manual

This manual is designed for Beginner to Intermediate learners targeting Data Engineering roles. It uses realistic banking datasets to teach production-ready SQL skills for ETL pipelines, analytics, and data warehousing.

Banking Dataset Context

The dataset simulates real banking systems including customer onboarding, account management, transaction processing, and foreign exchange reference data.

30 DAY SQL PRACTICE CHALLENGE

- Week 1: Basics & filtering
- Week 2: Joins & aggregations
- Week 3: CTEs & window functions
- Week 4: Performance & capstone

Core Tables Overview

Customers store onboarding information, accounts track balances and status, transactions capture financial activity, and FX rates provide reference exchange values.

PostgreSQL Installation

What is PostgreSQL?

PostgreSQL is an **open-source relational database system** used to store, manage, and query structured data.
Banks, fintechs, and enterprises use PostgreSQL for **transaction systems, data warehouses, and analytics platforms**.

Why Data Engineers Need This

- You must know how databases are **installed and accessed**

- You often set up **local environments** for testing ETL pipelines
- Understanding installation helps with **debugging connection issues**

Basic Installation (Conceptual)

- Install PostgreSQL server
- Install **pgAdmin** (GUI tool)
- PostgreSQL runs as a **service**
- pgAdmin is used to manage databases visually

💡 *In production, this may run on Linux servers or cloud platforms.*

Exercise

Task:

Install PostgreSQL locally and confirm:

- Server is running
- You can connect using pgAdmin

Creating a Database

What is a Database?

A database is a **logical container** that holds:

- Tables
- Schemas
- Views
- Functions
- Data

Think of it like a **folder** that contains all objects for one system.

Why It Matters

- Banks separate databases by **environment**:
  - dev
  - test

    - prod

- Data engineers often create databases for:
    - Data warehouses
    - Staging areas
    - Reporting systems

SQL Example

```
CREATE DATABASE banking_dw;
```

Exercise

Task:
Create a database called training_dw.
Goal:
Understand database-level isolation.

---

3 Creating Schemas

What is a Schema?

A schema is a **namespace inside a database** used to organize tables.

Example:

- raw.customers
- staging.transactions
- warehouse.fact_transactions

Why It Matters for Data Engineering

- Separates **raw data** from **cleaned data**
- Improves **security and organization**
- Standard practice in data warehouses

SQL Example

```
CREATE SCHEMA staging;

CREATE SCHEMA warehouse;
```

Exercise

Task:
Create schemas:

- raw
- staging
- warehouse

4 Creating Database Users (Roles)

What is a Database User? A database user (role) is an identity used to:

- Log in
- Run queries
- Access data

Why It Matters

- Data engineers **do not give everyone admin access**
- Different users have different permissions:
  - Read-only users
  - ETL service accounts
  - Admin users

SQL Example

```
CREATE USER reporting_user WITH PASSWORD 'StrongPassword123';
```

Exercise

Task:
Create a user called etl_user.

Goal:
Understand access separation.

---

5 Granting Access (Permissions)

What is GRANT? GRANT gives users permission to do something.

Examples:

- Read tables
- Insert data
- Create objects

Why It Matters in Banking

- Prevents **data leakage**
- Protects **financial data**
- Meets **audit and compliance requirements**

Grant Database Access

GRANT CONNECT ON DATABASE training_dw TO etl_user;

Grant Schema Access

GRANT USAGE ON SCHEMA staging TO etl_user;

Grant Table SELECT Access

GRANT SELECT ON ALL TABLES IN SCHEMA staging TO etl_user;

Exercise

Task:
Grant etl_user:

- Access to database
- SELECT on all tables in staging

Goal:
Practice controlled access.

---

6 Revoking Access

What is REVOKE?

REVOKE removes permissions previously granted.

Why It Matters

- Users change roles
- Access must be removed immediately
- Required for **security audits**

SQL Example

```
REVOKE SELECT ON ALL TABLES IN SCHEMA staging FROM etl_user;
```

Exercise

Task:
Revoke SELECT access from etl_user.

Goal:
Understand access control lifecycle.

---

7 Entity Relationship Diagrams (ERD)

What is an ERD?

An **Entity Relationship Diagram** visually shows:

- Tables (entities)
- Columns
- Primary keys
- Foreign keys
- Relationships

Why ERDs Matter for Data Engineers

- Helps understand **data flow**
- Prevents poor table design
- Essential for:
    - Data modeling
    - Warehousing
    - Analytics design

Example Relationships (Banking)

- One customer → Many accounts
- One account → Many transactions

```
customers (customer_id PK)
   |
   | 1-to-many
   |
accounts (account_no PK, customer_id FK)
   |
   | 1-to-many
   |
transactions (tran_id PK, account_no FK)
```

How to Create ERDs

- pgAdmin ERD tool
- draw.io
- dbdiagram.io
- Lucidchart

Database Schema (DDL)

Customers, Accounts, Transactions, and FX Rates tables are created using production-style PostgreSQL DDL scripts.

Customers Table

```
CREATE TABLE customers (
   customer_id BIGINT PRIMARY KEY,
   full_name   TEXT,
   email       TEXT,
   phone       TEXT,
   created_at  DATE
);
```

Accounts Table

```
CREATE TABLE accounts (
   account_no   TEXT PRIMARY KEY,
   customer_id  BIGINT,
   account_type TEXT,
   balance      NUMERIC(18,2) DEFAULT 0.00,
   status       TEXT,
   opened_date  DATE
);
```

FX Rates Table

```
CREATE TABLE fx_rates (
   ccy          TEXT,
   rate         NUMERIC(10,4),
   business_date DATE
);
```

Transactions Table

```
CREATE TABLE transactions (
```

```
    tran_id     BIGINT,
    account_no  TEXT,
    customer_id BIGINT,
    amount      NUMERIC(18,2),
    tran_type   TEXT,
    narrative   TEXT,
    tran_date   DATE,
    PRIMARY KEY (tran_id, tran_date)
) PARTITION BY RANGE (tran_date);
```

Automatic Daily Partitioning

The transactions table is partitioned by transaction date using RANGE partitioning. An automatic partition creation mechanism using PL/pgSQL functions and triggers ensures new daily partitions are created without manual intervention.

```
CREATE OR REPLACE FUNCTION create_daily_transaction_partition(p_tran_date DATE)
RETURNS VOID AS $$
DECLARE
    partition_name TEXT;
BEGIN
    partition_name := 'transactions_' || TO_CHAR(p_tran_date, 'YYYY_MM_DD');

    EXECUTE format(
        'CREATE TABLE IF NOT EXISTS %I PARTITION OF transactions
         FOR VALUES FROM (%L) TO (%L)',
        partition_name,
        p_tran_date,
        p_tran_date + INTERVAL '1 day'
    );
END;
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE FUNCTION trg_create_partition()
RETURNS TRIGGER AS $$
BEGIN
    PERFORM create_daily_transaction_partition(NEW.tran_date);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER before_insert_transactions
BEFORE INSERT ON transactions
FOR EACH ROW
EXECUTE FUNCTION trg_create_partition();
```

- ✔ **Fully automatic**
- ✔ **No insert failures**
- ✔ **Production-ready**

Indexing Strategy

Indexes are created on high-usage columns such as customer_id, account_no, transaction date, and business_date to improve query performance and ETL efficiency. Indexes improve query performance in high-volume banking systems:

```
CREATE INDEX idx_accounts_customer_id ON accounts(customer_id);

CREATE INDEX idx_transactions_account_no ON transactions(account_no);

CREATE INDEX idx_transactions_tran_date ON transactions(tran_date);

CREATE INDEX idx_fx_rates_business_date ON fx_rates(business_date);
```

Data Generation Strategy

Large-scale realistic datasets are generated using PostgreSQL generate_series and random functions to simulate millions of banking records for hands-on practice.

Generate Customers Data

```
INSERT INTO customers (customer_id, full_name, email, phone, created_at)
SELECT
  gs,
  'Customer ' || gs,
  'customer' || gs || '@bank.com',
  '080' || LPAD((RANDOM() * 10000000)::INT::TEXT, 7, '0'),
  CURRENT_DATE - (RANDOM() * 1500)::INT
FROM generate_series(1, 100000) gs;
```

Generate Fx Rates Data

```
INSERT INTO fx_rates (ccy, rate, business_date)
SELECT
ccy,
ROUND((CASE ccy
WHEN 'USD' THEN 1400 + RANDOM() * 100
WHEN 'EUR' THEN 1500 + RANDOM() * 100
WHEN 'GBP' THEN 1700 + RANDOM() * 100
WHEN 'NGN' THEN 1.00 END)::NUMERIC, 2),
```

```
CURRENT_DATE - gs
FROM generate_series(0, 60) gs,
(VALUES ('USD'),('EUR'),('GBP'),('NGN')) v(ccy);
```

Generate Accounts Data

```
INSERT INTO accounts (account_no, customer_id, account_type, balance, status, opened_date)
SELECT
'ACC' || customer_id || LPAD((RANDOM() * 2000999)::INT::TEXT, 3, '0'),
customer_id,
CASE
WHEN RANDOM() < 0.5 THEN 'SAVINGS'
WHEN RANDOM() < 0.8 THEN 'CURRENT'
ELSE 'DOMICILIARY'
END,
ROUND((RANDOM() * 5000000)::NUMERIC, 2),
CASE WHEN RANDOM() < 0.9 THEN 'ACTIVE' ELSE 'DORMANT' END,
CURRENT_DATE - (RANDOM() * 1200)::INT
FROM customers
WHERE RANDOM() < 0.8;
```

Generate transactions Data

```
INSERT INTO transactions (account_no, customer_id, amount, tran_type, narrative, tran_date)
SELECT
a.account_no,
a.customer_id,
ROUND((RANDOM() * 1000000)::NUMERIC, 2),
CASE WHEN RANDOM() < 0.5 THEN 'CREDIT' ELSE 'DEBIT' END,
CASE
WHEN RANDOM() < 0.3 THEN 'POS PURCHASE'
WHEN RANDOM() < 0.6 THEN 'ATM WITHDRAWAL'
ELSE 'TRANSFER'
END,
CURRENT_DATE - (RANDOM() * 90)::INT
FROM accounts a
WHERE RANDOM() < 0.05;
```

MODULE 1: SQL Fundamentals

Used to extract raw data from banking systems into ETL pipelines. Covers SELECT statements, filtering, ordering, and limiting results. These are foundational skills for extracting accurate data from banking systems.

Exercises

1. Retrieve all transactions above ₦50,000

2. Fetch account numbers and balances for ACTIVE accounts

Solutions

```
SELECT * FROM transactions WHERE amount > 50000;
SELECT account_no, balance
FROM accounts
WHERE status = 'ACTIVE';
```

MODULE 2: Joins

Joins are critical for combining normalized banking data. Focuses on INNER, LEFT, RIGHT, and FULL joins used to combine customers, accounts, and transactions into analytics-ready datasets.

Exercises

1. Get customer names and their transaction amounts

2. List customers with no transactions

Solutions

```
SELECT c.full_name, t.amount
FROM customers c
JOIN transactions t ON c.customer_id = t.customer_id;

SELECT c.customer_id, c.full_name
FROM customers c
LEFT JOIN transactions t ON c.customer_id = t.customer_id
WHERE t.customer_id IS NULL;
```

MODULE 3: Aggregations & KPIs

Teaches how to calculate totals, averages, and counts required for financial reporting and KPIs.

Exercises

1. Count transactions per account

2. Calculate daily transaction totals

Solutions

```
SELECT account_no, COUNT(*)
FROM transactions
GROUP BY account_no;

SELECT tran_date, SUM(amount)
FROM transactions
```

GROUP BY tran_date;

SQL PRACTICE ASSIGNMENT (Banking Dataset) BASIC DATA EXTRACTION

Tables Used

- customers
- accounts
- transactions
- fx_rates

Submission Guidelines

- Submit one or different consolidated **.sql file**
- Each query should be clearly commented
- **Deadline:** On or before Sunday. I need to see the script when next we meet, for you to run the query for me to see the query output for each task during the submission or meeting day.
- You can use AI tools such as ChatGPT for learning and guidance for better understanding.

🔹 SECTION A: Basic Data Extraction

1. Count the **total number of customers** in the database.
2. Count the **total number of accounts**.
3. Count the **total number of transactions**.
4. Retrieve records counts of fx_rates
5. Retrieve **customer ID, full name, phone, and email address** for customers whose date created is 1st January 2025.
6. Retrieve customers whose phone number is 0803499078
7. Retrieve customers whose email is customer10850@bank.com and customer50306500@bank.com
8. List 100 **active accounts** along with their current balances.
9. Retrieve 100 **savings accounts** with balances **greater than ₦500,000**.
10. List the account types that exist in the database.
11. Display the **100 transaction records**.
12. Retrieve 500 transactions with amounts **greater than ₦100,000**.
13. Display all **debit transactions** that occurred within the month of December 2025.
14. Help me get account details for account no ACC5273, it should show account name, email, opening date, status, phone, account type, customer id.
15. Help me get all account transaction details for the **account** ACC733869 in the past 3 months.
16. it should show account name, email, opening date, status, phone, account type, customer id, transaction id, transaction type, amount, transaction date, narrative.
17. Get transactions for account ACC1117425 with ATM narrative in the month of June 2025.

18. List 329 customers who do not own any accounts.
19. Show transaction ID, account type, and transaction amount for customer with id 1247 on the 12th and 20th of october 2025.
20. Retrieve 10 customers who own more than one account.
21. Display all transactions performed on current accounts in the month of June 2025.
22. List 500 customers whose account balance exceeds ₦1,000,000.
23. Display transaction details alongside their **USD FX rate** for the transaction date as of 7th January 2026 for account ACC5055287.
24. Get the transaction details for ACC11548604 in Jan 2026 and Convert all transaction amounts to USD using the fx_rates table as of 6th Jan 2026.
25. Calculate the total transaction amount per account in the month of January 2026.
26. Calculate the total number of transactions per customer in the month of January 2026.
27. Display the top 5 customers by total transaction value in the month of December 2025.
28. Calculate the average transaction amount for each customer between October 2025 and December 2025.
29. Display the maximum transaction and amount for account ACC1251635947.
30. Calculate the total transaction Credit turnover for all Savings accounts in the year 2025.
31. Calculate the total transaction Debit turnover for all Current accounts in the year 2025.

MODULE 4: SUBQUERIES & CTEs

CTE Task: High-Value Customer Activity Report

Question: Identify the top 5 customers who have the highest total transaction volume (sum of all their transactions) across all their accounts. The result should show the Customer Name, their Total Number of Accounts, and the Grand Total Transaction Amount.

Why use a CTE here?

1. **Readability:** Instead of one giant query with multiple nested subqueries, we define "building blocks" (CustomerAccountCounts and CustomerTransactionTotals) first.
2. **Organization:** It separates the logic of *counting accounts* from the logic of *summing money*, making it easier to debug if the numbers look wrong.
3. **Performance:** In many PostgreSQL scenarios, CTEs help the optimizer organize the execution plan more efficiently for complex joins.

Solutions

```
WITH CustomerAccountCounts AS (
  -- CTE 1: Count accounts per customer
  SELECT
    customer_id,
    COUNT(account_number) AS total_accounts
  FROM accounts
  GROUP BY customer_id
```

```
),
CustomerTransactionTotals AS (
   -- CTE 2: Sum all transactions per customer across all their accounts
   SELECT
      a.customer_id,
      SUM(t.amount) AS grand_total_volume
   FROM accounts a
   JOIN transactions t ON a.account_number = t.account_number
   GROUP BY a.customer_id
)
-- Main Query: Combine the CTEs and get customer names
SELECT
   c.full_name,
   cac.total_accounts,
   ctt.grand_total_volume
FROM customers c
JOIN CustomerAccountCounts cac ON c.customer_id = cac.customer_id
JOIN CustomerTransactionTotals ctt ON c.customer_id = ctt.customer_id
ORDER BY ctt.grand_total_volume DESC
LIMIT 5;
```

MODULE 5: Window Functions

Exercises

1. Calculate running balance per account

2. Rank transactions by amount per account

Solutions

```
SELECT account_no, tran_date, amount,
SUM(amount) OVER (PARTITION BY account_no ORDER BY tran_date)
FROM transactions;

SELECT account_no, amount,
RANK() OVER (PARTITION BY account_no ORDER BY amount DESC)
FROM transactions;
```

MODULE 6: DATA CLEANING & TRANSFORMATION

```
UPDATE customers SET email = LOWER(email);

SELECT TRIM(narrative) FROM transactions;
```

MODULE 7: INDEXES & PERFORMANCE

```
EXPLAIN ANALYZE SELECT * FROM transactions WHERE tran_date = '2025-01-01';
```

MODULE 8: TRANSACTIONS & DATA SAFETY

```
BEGIN;

UPDATE accounts SET balance = balance - 1000 WHERE account_no = '123';

UPDATE accounts SET balance = balance + 1000 WHERE account_no = '456';

COMMIT;
```

MODULE 9: VIEWS & MATERIALIZED VIEWS

```
CREATE MATERIALIZED VIEW mv_daily_transactions AS

SELECT tran_date, SUM(amount) total

FROM transactions GROUP BY tran_date;
```

MODULE 10: STORED PROCEDURES & FUNCTIONS

```
CREATE OR REPLACE FUNCTION get_total_amount(acc_no TEXT)

RETURNS NUMERIC AS $$

BEGIN

RETURN (SELECT SUM(amount) FROM transactions WHERE account_no = acc_no);

END;

$$ LANGUAGE plpgsql;
```

Capstone Project

Build a mini reporting data warehouse by cleaning transactions, aggregating daily KPIs, creating optimized views, and validating performance.

DATA ENGINEERING INTERVIEW SQL

Final Advice

If you can confidently write complex joins, use window functions, and optimize SQL queries, you are production-ready for Data Engineering roles.